

ELL782: Computer Architecture

Assignment-1: Encryption, Decryption, and Key-Based Hashing

Group C

Released: August 7, 2025
Submission Deadline: September 7, 2025

1 Introduction

In this assignment, you will design and implement hardware modules for encryption, decryption, and key-based hashing, which are critical components in secure communication systems. The assignment consists of two parts: Part A (Encryption Module) and Part B (Decryption and Hash Module). You will implement the modules in Verilog, simulate them, and verify correctness using a verification module. The assignment must be completed individually.

1.1 Precise Algorithm Scenario: Single-Round SPN

You will implement a simplified **substitution-permutation network (SPN)** cipher for an 8-bit data block. The process is as follows:

1. **SubBytes (S-box)**: Map each 8-bit input byte through a fixed substitution box (S-box). The full S-box and inverse S-box tables are provided and must be implemented as LUTs or ROMs in your code.
2. **PermuteBits**: Perform a fixed bit permutation defined as swapping the **upper nibble (bits 7–4)** and the **lower nibble (bits 3–0)** of the byte. That is:

$$\text{PermuteBits}(b_7b_6b_5b_4\ b_3b_2b_1b_0) = b_3b_2b_1b_0\ b_7b_6b_5b_4$$

For example, if the input is `0xD7` (binary `11010111`), then:

$$\text{PermuteBits}(0xD7) = 0x7D \quad (\text{binary } 01111101)$$

3. **AddRoundKey**: XOR the permuted byte with the fixed 8-bit key $K = 0x5A$.

Encryption Equation

$$\text{Encrypted} = \text{AddRoundKey}(\text{PermuteBits}(\text{SBox}(\text{Input}))) = \text{PermuteBits}(\text{SBox}(\text{Input})) \oplus K$$

Decryption Equation

To decrypt, apply inverse operations in reverse order:

$$\text{Decrypted} = \text{SBox}^{-1}(\text{PermuteBits}^{-1}(\text{Encrypted} \oplus K))$$

where PermuteBits^{-1} swaps the nibbles back to the original order.

Worked Example: Encryption

Using the provided S-box:

$$\text{SBox}(0x41) = 0xD7$$

Stepwise encryption of input byte $0x41$:

- Input: $0x41$ (binary: 01000001)
- $\text{SBox}(0x41) = 0xD7$ (binary: 11010111)
- $\text{PermuteBits}(0xD7) = 0x7D$ (binary: 01111101)
- $0x7D \oplus 0x5A = 0x27$ (binary: 00100111)

Final encrypted output: $0x27$.

Worked Example: Decryption

Given encrypted byte $0x27$:

- XOR with key: $0x27 \oplus 0x5A = 0x7D$ (binary: 01111101)
- Inverse permute: Swap nibbles of $0x7D$ gives $0xD7$ (binary: 11010111)
- Apply inverse S-box: $\text{SBox}^{-1}(0xD7) = 0x41$

Recovered original input: $0x41$.

1.2 Key-Based Hash Function

Compute an 8-bit hash as:

$$\text{hash} = ((\text{Encrypted} \oplus K) \ll 2) \bmod 256$$

where $\ll 2$ is a 2-bit left shift with overflow bits discarded.

Example using the previous encrypted value $0x27$:

$$0x27 \oplus 0x5A = 0x7D, \quad 0x7D \ll 2 = 0xF4 \quad \Rightarrow \quad \text{hash} = 0xF4$$

—

2 Part A: Encryption Module

Design an 8-bit encryption module implementing the above single-round SPN. Your module must:

- Accept an 8-bit input data and use the fixed key $K = 0x5A$.
- Implement the S-box lookup using a LUT or ROM.
- Perform the permutation by swapping the byte's nibbles.
- XOR the permuted output with the key.
- Output the 8-bit encrypted data.
- Work combinationally or pipelined, as per your design.

2.1 Tasks

1. Implement the encryption module in Verilog.
2. Create a self-checking testbench with the provided test cases and at least one additional test case. Use assertions to verify correctness.
3. Synthesize your design using a standard synthesis tool. Submit area and critical path delay reports.

2.2 Deliverables

- Verilog code for encryption and testbench.
 - Simulation waveforms and reports showing input/output correctness (in binary, with pass/fail).
 - Synthesis report (area, delay).
-

3 Part B: Decryption and Hash Module

Implement a decryption module that reverses the encryption operation and a hash module that computes the key-based hash from the encrypted data. Also implement a verification module that:

- Re-encrypts the decrypted data and compares it with the original encrypted input.
- Compares the computed hash with the reference hash.
- Outputs flags indicating a match or mismatch.

3.1 Tasks

1. Implement the decryption module in Verilog; apply inverse key XOR, inverse permutation, and inverse S-box.
2. Implement the hash function module as specified.
3. Implement a verification module with self-checking capability.
4. Simulate the modules with provided and new test cases.
5. Synthesize the entire design; report area, delay, and throughput (cycles/op).
6. Analyze design trade-offs in your report.

3.2 Deliverables

- Verilog code for decryption, hash, verification, and testbenches.
 - Simulation results (binary outputs with verification flags).
 - Synthesis reports.
 - A report describing design choices, verification, and trade-offs.
-

4 Test Cases

4.1 S-box and Inverse S-box Tables

The full S-box and inverse S-box tables will be provided on the course website or can be hardcoded from the provided data files.

4.2 Test Case 1 (Encryption)

- Input: 0x41, Key: 0x5A
- Expected Encrypted Output: 0x27 (from worked example)

4.3 Test Case 2 (Encryption)

- Input: 0xFF, Key: 0x5A
- Expected Encrypted Output: (Compute using your S-box and permutation)

4.4 Test Case 3 (Decryption and Hash)

- Input: Encrypted data 0x27, Key: 0x5A
- Expected Decrypted Output: 0x41
- Expected Valid Flag: 1
- Expected Hash: 0xF4
- Expected Match Flag: 1

4.5 Additional Test Case

Generate a custom sequence of 10 random 8-bit inputs. Use a Python script to generate expected encrypted, decrypted, and hash values and verify correctness of your hardware modules.

5 Report

Your report should include:

- Detailed design description and architecture diagrams (block diagrams showing data flow and control).
- Implementation details (LUTs, combinational or pipelined logic).
- Simulation and synthesis results, including area, delay, and throughput.
- Verification methodology and testbench description.
- Discussion of trade-offs (e.g., LUT vs. combinational logic, latency vs. complexity).
- Any issues or common pitfalls encountered and how you addressed them.
- Optional optimizations or additional features implemented.

Format: PDF, max 10 pages, named `report.pdf`.

6 Student Checklist

Implemented S-box and inverse S-box accurately.

Correctly implemented nibble swapping for permutation and inverse.

Properly applied XOR with fixed key 0x5A.

Testbenches include self-checking assertions with expected outputs.

Binary output reports include valid/match flags.

Synthesis reports submitted with area and delay.

Block diagrams and error analysis included in the report.

Submitted all required files zipped and named as specified.

—

7 Common Pitfalls

- Confusing nibble order in permutation.
- Incorrect or incomplete S-box tables.
- Failing to apply inverse operations in correct order.
- Missing self-checking mechanisms in testbenches.
- Incorrect bit-width handling (should be 8 bits throughout).
- Neglecting valid and match signal logic.

—

8 Submission Guidelines

- Submit a single zip file on Moodle named `Assignment4_<EntryNumber>.zip`, containing:
 - Verilog source files (`encrypt.v`, `decrypt.v`, `hash.v`, `verify.v`).
 - Testbenches with simulation waveforms or output reports.
 - Synthesis reports.
 - Report (`report.pdf`).
 - Python scripts to generate test vectors and verify results (optional but encouraged).
- We will run plagiarism detection (MOSS). Cheating will be penalized as per course policy.
- Your solution will be tested with additional hidden test vectors.
- Demos will be conducted for the assignment.
- Submission deadline: September 7, 2025, 23:59 IST.